OPTIMA: Optimal Provable Task-Scheduling in Asymmetric Multiprocessors via Weighted Shortest Processing Time and Branch-and-Bound eBPF Extensions

Risheendra MN

Abstract*—Asymmetric multiprocessor architectures, comprising high-throughput Performance (P) cores and energy-efficient Efficiency (E) cores with disparate compute capacities, present fundamental challenges for general-purpose OS schedulers. Existing kernel schedulers rely heavily on empirical heuristics that lack formal optimality guarantees and struggle under mixed latency-critical and compute-intensive workloads. In this paper, we develop the complete formal mathematical foundations of OPTIMA, a provably optimal task-scheduling framework implemented via Linux `sched_ext` eBPF. We formalize the asymmetric scheduling problem under a dual objective: minimizing total weighted completion time $\sum w_j C_j$ while guaranteeing hard real-time deadline constraints $C_j \leq d_j$. We prove that density-based Weighted Shortest Processing Time (WSPT) job sequencing via Smith's Rule, combined with continuous Dantzig knapsack partitioning, achieves optimal weight depletion across asymmetric cores. Furthermore, we formulate an online Branch-and-Bound preemption bounding framework with an admissible lower bound $LB(S)$, proving that preempting under $\Delta\text{Cost} < 0$ bounds the competitive ratio to $(1+\epsilon)$ against an offline optimum. Finally, using a quadratic Lyapunov drift formulation, we prove that OPTIMA's adaptive quantum scaling and hierarchical work-stealing maintain strong queue stability and starvation freedom under all admissible workloads. Complete algorithmic specifications in kernel eBPF hot-paths are provided.

I. System Model and Problem Formulation

A. Heterogeneous Multiprocessor Architecture

We model a heterogeneous asymmetric multiprocessor system as a set of M discrete computing cores denoted by:

$$\mathcal{M} = \mathcal{P} \cup \mathcal{E}, \quad \mathcal{P} \cap \mathcal{E} = \emptyset \tag{1}$$

where $\mathcal{P} = \{C_1, C_2, \ldots, C_{M_P}\}$ represents the partition of $M_P = |\mathcal{P}|$ Performance cores (P-cores), and $\mathcal{E} = \{C_{M_P+1}, C_{M_P+2}, \ldots, C_{M_P+M_E}\}$ represents the partition of $M_E = |\mathcal{E}|$ Efficiency cores (E-cores), such that the total core count is $M = M_P + M_E$.

Each core $C_k \in \mathcal{M}$ is characterized by an intrinsic, time-invariant computational capacity (effective instruction processing frequency) $c_k \in \mathbb{R}_{>0}$, defined as:

$$c_k = \begin{cases} c_P, & \forall C_k \in \mathcal{P} \\ c_E, & \forall C_k \in \mathcal{E} \end{cases} \tag{2}$$

where $c_P > c_E > 0$. We define the asymmetric processing speedup ratio as:

$$\mu \triangleq \frac{c_P}{c_E} > 1 \tag{3}$$

The memory and cache subsystem is structured hierarchically: P-cores in $\mathcal{P}$ share a common coherent L3 cache domain, while E-cores in $\mathcal{E}$ reside in an independent L3 cache domain. Cores may optionally support Simultaneous Multithreading (SMT), where physical core C_k provides two sibling hardware thread contexts.

B. Dynamic Online Task Model

The workload submitted to the operating system is represented as a dynamic, continuous online sequence of tasks:

$$\mathcal{J} = \{J_1, J_2, \ldots, J_n, \ldots\} \tag{4}$$

Each task $J_j \in \mathcal{J}$ is formally specified by a 5-tuple:

$$J_j = (r_j,\, p_j,\, w_j,\, d_j,\, \mathcal{T}_j) \tag{5}$$

where:

1) $r_j \in \mathbb{R}_{\geq 0}$ denotes the release (arrival or wakeup) timestamp of task J_j.
2) $p_j \in \mathbb{R}_{>0}$ represents the nominal processing requirement (computational work volume in cycles or reference execution time measured on a baseline core of capacity c_E). In the kernel implementation, p_j is estimated via an Exponentially Weighted Moving Average (EWMA) of past burst durations.
3) $w_j \in [w_{\min}, w_{\max}] \subset \mathbb{R}_{>0}$ denotes the task priority weight, derived from POSIX nice levels via $w_j = 100 \times 1.25^{-\text{nice}_j}$.
4) $d_j \in \mathbb{R}_{>0} \cup \{\infty\}$ is the absolute completion deadline. Tasks are partitioned into real-time/interactive tasks $\mathcal{J}_{\text{RT}} \triangleq \{J_j \in \mathcal{J} \mid d_j < \infty\}$ and compute-bound batch tasks $\mathcal{J}_{\text{Batch}} \triangleq \{J_j \in \mathcal{J} \mid d_j = \infty\}$.
5) $\mathcal{T}_j \in \{\text{Interactive}, \text{Batch}\}$ designates the behavioral execution classification.

Definition 1 (Task Density)**.** *For any task $J_j \in \mathcal{J}$, its task density ρ_j is defined as the ratio of its priority weight to its nominal execution requirement:*

$$\rho_j \triangleq \frac{w_j}{p_j} \tag{6}$$

representing the marginal priority weight delivered per unit of computational work.

C. Execution Capacity Scaling

The physical execution duration of task J_j when dispatched to processor core $C_k \in \mathcal{M}$ depends inversely on the processing capacity c_k:

$$p_{j,k} = \frac{p_j}{c_k} = \begin{cases} \dfrac{p_j}{c_P}, & C_k \in \mathcal{P} \\ \dfrac{p_j}{c_E} = \mu \cdot \dfrac{p_j}{c_P}, & C_k \in \mathcal{E} \end{cases} \tag{7}$$

Thus, executing task J_j on a Performance core reduces its latency by a factor of μ relative to execution on an Efficiency core.

D. Dual-Objective Optimization Formulation

Let $S_j \geq r_j$ and C_j denote the scheduling start time and final completion time of task J_j under schedule $\mathcal{S}$, respectively. If task J_j undergoes preemptive execution across time slots, its total elapsed execution satisfies:

$$\sum_{k \in \mathcal{M}} c_k \int_{S_j}^{C_j} \mathbf{1}_{\{J_j \text{ runs on } C_k \text{ at time } t\}}\, dt = p_j \tag{8}$$

The primary objective is to minimize the total weighted completion time across all tasks while strictly satisfying real-time deadline guarantees for interactive and audio tasks:

$$\min_{\mathcal{S}} \quad \Phi(\mathcal{S}) \triangleq \sum_{J_j \in \mathcal{J}} w_j C_j(\mathcal{S}) \tag{9}$$

$$\text{subject to} \quad C_j(\mathcal{S}) \leq d_j, \quad \forall J_j \in \mathcal{J}_{\mathrm{RT}} \tag{10}$$

$$S_j \geq r_j, \quad \forall J_j \in \mathcal{J} \tag{11}$$

$$\sum_{J_j \in \mathcal{J}} \mathbf{1}_{\{J_j \text{ on } C_k \text{ at } t\}} \leq 1, \quad \forall C_k \in \mathcal{M},\ t \geq 0 \tag{12}$$

Constraint (10) enforces real-time feasibility, Constraint (11) enforces causality, and Constraint (12) guarantees non-overlapping single-task execution per physical processing core.

II. Optimality of Density-Based WSPT under Asymmetric Processing Capacity

A. Smith's Rule Optimality on Single-Core Dedicated Queues

We begin by establishing the fundamental optimality of density-ordered execution on an isolated processing core.

Lemma 1 (Optimality of Density-Based WSPT (Smith's Rule))**.** *Let $\mathcal{J}_k = \{J_1, J_2, \ldots, J_n\}$ be a set of n tasks available simultaneously at time t_0 on a single processor core C_k with compute capacity c_k. Any non-preemptive schedule that orders tasks in non-increasing order of density:*

$$\rho_{(1)} \geq \rho_{(2)} \geq \cdots \geq \rho_{(n)} \tag{13}$$

strictly minimizes the total weighted completion time $\sum_{j=1}^{n} w_j C_j$.

Proof. We proceed via an adjacent pairwise exchange (interchange) argument. Assume for contradiction that there exists an optimal schedule $\mathcal{S}^*$ that does not follow the non-increasing density ordering.

Because the density ordering is not satisfied everywhere in $\mathcal{S}^*$, there must exist at least two adjacent tasks, say J_a immediately followed by J_b, such that:

$$\rho_a < \rho_b \iff \frac{w_a}{p_a} < \frac{w_b}{p_b} \iff w_a p_b < w_b p_a \tag{14}$$

Let $t \geq t_0$ denote the start time of task J_a under schedule $\mathcal{S}^*$. Under $\mathcal{S}^*$, task J_a executes during $[t,\, t + p_{a,k})$, and task J_b executes during $[t + p_{a,k},\, t + p_{a,k} + p_{b,k})$. Their respective completion times are:

$$C_a(\mathcal{S}^*) = t + \frac{p_a}{c_k} \tag{15}$$

$$C_b(\mathcal{S}^*) = t + \frac{p_a}{c_k} + \frac{p_b}{c_k} \tag{16}$$

The combined contribution of J_a and J_b to the objective function $\Phi(\mathcal{S}^*)$ is:

$$\begin{aligned} \Phi_{a,b}(\mathcal{S}^*) &= w_a C_a(\mathcal{S}^*) + w_b C_b(\mathcal{S}^*) \\ &= w_a \left(t + \frac{p_a}{c_k} \right) + w_b \left(t + \frac{p_a + p_b}{c_k} \right) \\ &= (w_a + w_b)t + \frac{1}{c_k}\Big(w_a p_a + w_b p_a + w_b p_b \Big) \end{aligned} \tag{17}$$

Now, construct an alternative schedule $\mathcal{S}'$ by interchanging the execution positions of J_a and J_b, while preserving the positions of all tasks scheduled prior to time t and all tasks scheduled after time $t + \frac{p_a + p_b}{c_k}$.

Under schedule $\mathcal{S}'$, task J_b executes first during $[t,\, t + p_{b,k})$, followed by task J_a during $[t + p_{b,k},\, t + p_{b,k} + p_{a,k})$. Their completion times under $\mathcal{S}'$ become:

$$C_b(\mathcal{S}') = t + \frac{p_b}{c_k} \tag{18}$$

$$C_a(\mathcal{S}') = t + \frac{p_b}{c_k} + \frac{p_a}{c_k} \tag{19}$$

The combined contribution of J_a and J_b under $\mathcal{S}'$ is:

$$\begin{aligned} \Phi_{a,b}(\mathcal{S}') &= w_b C_b(\mathcal{S}') + w_a C_a(\mathcal{S}') \\ &= w_b \left(t + \frac{p_b}{c_k} \right) + w_a \left(t + \frac{p_b + p_a}{c_k} \right) \\ &= (w_a + w_b)t + \frac{1}{c_k}\Big(w_b p_b + w_a p_b + w_a p_a \Big) \end{aligned} \tag{20}$$

For all tasks $J_j \notin \{J_a, J_b\}$, their completion times are completely unaffected:

1) Tasks completed before time t finish before the interchange occurs, so $C_j(\mathcal{S}') = C_j(\mathcal{S}^*)$.
2) Tasks scheduled after J_a and J_b begin at $t + \frac{p_b + p_a}{c_k} = t + \frac{p_a + p_b}{c_k}$. Because the total execution duration of $\{J_a, J_b\}$ is invariant under permutation, the start and completion times of all subsequent tasks remain identical: $C_j(\mathcal{S}') = C_j(\mathcal{S}^*)$ for all subsequent j.

Now, evaluate the net cost difference $\Delta\Phi \triangleq \Phi(\mathcal{S}') - \Phi(\mathcal{S}^*)$:

$$\begin{aligned} \Delta\Phi &= \Phi_{a,b}(\mathcal{S}') - \Phi_{a,b}(\mathcal{S}^*) \\ &= \frac{1}{c_k}\Big(w_b p_b + w_a p_b + w_a p_a - w_a p_a - w_b p_a - w_b p_b \Big) \\ &= \frac{1}{c_k}\Big(w_a p_b - w_b p_a \Big) \end{aligned} \tag{21}$$

From the inversion condition in (14), $w_a p_b - w_b p_a < 0$. Since $c_k > 0$, it follows that:

$$\Delta\Phi = \frac{1}{c_k}(w_a p_b - w_b p_a) < 0 \implies \Phi(\mathcal{S}') < \Phi(\mathcal{S}^*) \quad (22)$$

This strictly contradicts the assumption that $\mathcal{S}^*$ is an optimal schedule. Since the permutation group S_n is finite and generated by adjacent transpositions, any schedule can be transformed into non-increasing density order via a finite sequence of adjacent exchanges, with each exchange strictly decreasing the objective function until no inversions remain. Thus, Smith's density ordering is uniquely optimal. □

B. Extension to Asymmetric Dual-Speed Multiprocessors

We now extend density-based scheduling to asymmetric multiprocessors $\mathcal{M} = \mathcal{P} \cup \mathcal{E}$ with $c_P > c_E$.

Theorem 1 (Optimality of Density-Tiered Asymmetric Processor Partitioning)**.** *Let $\mathcal{M} = \mathcal{P} \cup \mathcal{E}$ be an asymmetric multiprocessor system with capacities $c_P > c_E$. Under any concurrent assignment of tasks to Performance and Efficiency cores, assigning tasks with higher density ρ_j to Performance cores strictly maximizes the instantaneous rate of remaining weight reduction and minimizes total weighted completion time.*

Proof. Let $W(t) \triangleq \sum_{J_j \in \mathcal{Q}(t)} w_j$ denote the total unscheduled weight in the system at time t, where $\mathcal{Q}(t)$ is the set of active, uncompleted tasks. Using the integral representation of total weighted completion time:

$$\sum_{J_j \in \mathcal{J}} w_j C_j = \int_0^\infty W(t)\, dt \quad (23)$$

To minimize $\int_0^\infty W(t)\, dt$, the scheduler must maximize the instantaneous rate of weight decay $-\dot{W}(t)$ at every time t.

At any time t, let $J_{\sigma(k)}$ be the task currently executing on core $C_k \in \mathcal{M}$. The rate of computational progress delivered to task $J_{\sigma(k)}$ is $c_k/p_{\sigma(k)}$. Because task $J_{\sigma(k)}$ completes after executing $p_{\sigma(k)}$ work, the effective rate of weight retirement delivered by core C_k is:

$$-\dot{W}_k(t) = w_{\sigma(k)} \cdot \frac{c_k}{p_{\sigma(k)}} = \rho_{\sigma(k)} \cdot c_k \quad (24)$$

Summing over all M cores, the total instantaneous rate of weight depletion is:

$$-\dot{W}(t) = \sum_{k=1}^{M} \rho_{\sigma(k)} c_k = \sum_{k \in \mathcal{P}} \rho_{\sigma(k)} c_P + \sum_{k \in \mathcal{E}} \rho_{\sigma(k)} c_E \quad (25)$$

Now, consider any two tasks J_1 and J_2 running concurrently, with J_1 assigned to a P-core (c_P) and J_2 assigned to an E-core (c_E). Suppose for contradiction that $\rho_1 < \rho_2$. The weight depletion rate under this assignment $\mathcal{A}$ is:

$$R(\mathcal{A}) = \rho_1 c_P + \rho_2 c_E \quad (26)$$

Interchanging the cores of J_1 and J_2 produces assignment $\mathcal{A}'$ with depletion rate:

$$R(\mathcal{A}') = \rho_2 c_P + \rho_1 c_E \quad (27)$$

Computing the difference:

$$\begin{aligned} R(\mathcal{A}') - R(\mathcal{A}) &= (\rho_2 c_P + \rho_1 c_E) - (\rho_1 c_P + \rho_2 c_E) \\ &= (\rho_2 - \rho_1)(c_P - c_E) \end{aligned} \quad (28)$$

Because $\rho_2 > \rho_1$ and $c_P > c_E$, we have $(\rho_2 - \rho_1)(c_P - c_E) > 0$, which implies $R(\mathcal{A}') > R(\mathcal{A})$.

More generally, by the Hardy-Littlewood-Pólya Rearrangement Inequality, for any two sequences of real numbers sorted in non-increasing order:

$$\rho_{(1)} \geq \rho_{(2)} \geq \cdots \geq \rho_{(M)} \quad (29)$$

$$c_{(1)} \geq c_{(2)} \geq \cdots \geq c_{(M)} \quad (30)$$

the inner product $\sum_{k=1}^{M} \rho_{\sigma(k)} c_k$ is strictly maximized if and only if the permutation σ matches the monotonic ordering, i.e., $\sigma(k) = (k)$. Because $c_k = c_P$ for all $k \in \mathcal{P}$ and $c_k = c_E$ for all $k \in \mathcal{E}$ with $c_P > c_E$, maximum weight depletion requires:

$$\min_{k \in \mathcal{P}} \rho_{\sigma(k)} \geq \max_{k' \in \mathcal{E}} \rho_{\sigma(k')} \quad (31)$$

Therefore, allocating tasks with the highest densities ρ_j to Performance cores maximizes the instantaneous reduction of system backlog, minimizing total weighted completion time. □

C. Continuous Dantzig Knapsack Formulation in eBPF

When tasks wake up dynamically, full combinatorial reassignment across all cores is computationally prohibitive. OPTIMA formulates core assignment as a continuous knapsack problem. Let $x_j \in \{0, 1\}$ denote whether task J_j is allocated to a Performance core ($x_j = 1$) or an Efficiency core ($x_j = 0$). The latency reduction gain obtained by assigning J_j to a P-core instead of an E-core is:

$$\text{Gain}(J_j) = w_j \left(\frac{p_j}{c_E} - \frac{p_j}{c_P} \right) = w_j p_j \left(\frac{c_P - c_E}{c_P c_E} \right) \quad (32)$$

Subject to the P-core capacity constraint $\sum_j x_j \leq M_P$, the continuous relaxation upper bound (Dantzig-Wolfe relaxation) ranks tasks by marginal gain per unit capacity. By (32), this marginal efficiency is directly proportional to task density $\rho_j = w_j/p_j$. OPTIMA implements this continuous frontier in kernel eBPF by defining a critical density threshold ρ^*: tasks with $\rho_j \geq \rho^*$ or $w_j \geq w_{\text{RT}}$ are routed to $\mathcal{P}$, while batch tasks with $\rho_j < \rho^*$ are placed on $\mathcal{E}$ when P-core queues are saturated. This yields a deterministic $\mathcal{O}(1)$ kernel hot-path decision while tracking the optimal continuous relaxation.

III. Branch-and-Bound Online Preemption Bounding

A. The Online Preemption Decision Problem

In an asymmetric multiprocessing system, when a high-priority interactive or real-time task $J_{\text{wake}} \in \mathcal{J}_{\text{RT}}$ arrives at time t, all Performance cores $\mathcal{P}$ may be actively executing batch tasks. The scheduler faces an immediate online decision:

- **Option 1 (Non-Preemptive Wait):** Enqueue J_{wake} into the dispatch queue and allow currently running tasks to execute until their time quanta expire.

- **Option 2 (Preemptive Displacement):** Select an active P-core $c^* \in \mathcal{P}$, immediately preempt its currently running task $J_{\text{curr}}(c^*)$, and schedule J_{wake} on core c^*.

Preemption is not cost-free: preempting a task introduces a context-switch and hardware cache invalidation penalty $C_{\text{switch}} > 0$, while delaying the running task J_{curr} by at least the duration of J_{wake}'s execution slice.

B. Derivation of the Mathematical Lower Bound $LB(S)$

For each candidate Performance core $c \in \mathcal{P}$, let $J_{\text{curr}}(c)$ denote the task currently running on core c. Let:

- $w_{\text{curr}}(c)$ be the priority weight of $J_{\text{curr}}(c)$.
- $t_{\text{start}}(c)$ be the timestamp when $J_{\text{curr}}(c)$ commenced its current time slice.
- $\tau(c)$ be the allocated quantum of $J_{\text{curr}}(c)$.
- $R(c) \triangleq \max\Big(0,\, \tau(c) - \big(t - t_{\text{start}}(c)\big)\Big)$ be the remaining slice duration of $J_{\text{curr}}(c)$.
- w_{wake} be the priority weight of waking task J_{wake}.

Definition 2 (Preemption Penalty and Gain)**.** *For any candidate core $c \in \mathcal{P}$:*

$$\text{Penalty}(c) \triangleq w_{\text{curr}}(c) \cdot R(c) + C_{\text{switch}} \tag{33}$$

$$\text{Gain}(c) \triangleq w_{\text{wake}} \cdot R(c) \tag{34}$$

The net latency cost differential incurred by preempting core c instead of waiting is:

$$\begin{aligned}\Delta\text{Cost}(c) &\triangleq \text{Penalty}(c) - \text{Gain}(c) \\ &= \Big(w_{\text{curr}}(c) - w_{\text{wake}}\Big) \cdot R(c) + C_{\text{switch}}\end{aligned} \tag{35}$$

We formulate the state space as a search tree S, where each candidate branch corresponds to selecting core $c \in \mathcal{P}$ for preemption, and the root represents the null action (no preemption).

Definition 3 (Admissible Lower Bound Function)**.** *For any subtree rooted at candidate core $c \in \mathcal{P}$, the bounding function is defined as:*

$$LB(c) \triangleq \Big(w_{\text{curr}}(c) - w_{\text{wake}}\Big) \cdot R(c) + C_{\text{switch}} \tag{36}$$

C. Branch-and-Bound Pruning Invariants

OPTIMA evaluates $LB(c)$ across candidate cores using four rigorous pruning rules:

1) **Rule 1 (Trivial Optimum / Idle Core):** If candidate core c is idle ($w_{\text{curr}}(c) = 0$), then $\text{Penalty}(c) = 0$ (no running task is displaced), yielding:
$$LB(c) = -w_{\text{wake}} \cdot R(c) < 0 \tag{37}$$
This constitutes the absolute minimum bound; the search terminates immediately with $c^* = c$ in $\mathcal{O}(1)$ time.
2) **Rule 2 (Inversion Safety Pruning):** If $w_{\text{curr}}(c) \geq w_{\text{wake}}$, preempting core c would displace a higher- or equal-priority task to run a lower-priority task, inducing priority inversion. Because $\big(w_{\text{curr}}(c) - w_{\text{wake}}\big) \geq 0$ and $C_{\text{switch}} > 0$:
$$LB(c) \geq C_{\text{switch}} > 0 \tag{38}$$
The branch cannot yield a net cost reduction. It is strictly inadmissible and pruned.
3) **Rule 3 (Granularity Threshold Pruning):** If the remaining quantum $R(c) < \delta_{\min}$, where:
$$\delta_{\min} \triangleq \frac{C_{\text{switch}}}{w_{\text{wake}} - w_{\text{curr}}(c)} \tag{39}$$
the execution gain $(w_{\text{wake}} - w_{\text{curr}})R(c)$ is strictly insufficient to amortize the hardware context-switch penalty C_{switch}. Thus $LB(c) > 0$, and the branch is safely pruned.
4) **Rule 4 (Incumbent Cutoff):** Maintain the best upper bound $UB_{\text{best}} \triangleq \min_{j \in \text{visited}} LB(j)$, initialized to 0. If $LB(c) \geq UB_{\text{best}}$, candidate core c cannot improve upon the best incumbent preemption target. The branch is pruned.

Proposition 1 (Admissibility of Pruning)**.** *The bounding function $LB(c)$ is admissible. Pruning any branch where $LB(c) \geq 0$ or $LB(c) \geq UB_{\text{best}}$ guarantees that no globally optimal preemption target is eliminated.*

Proof. Let c^* be the true optimal preemption target. If preempting c^* is optimal, it must satisfy $\Delta\text{Cost}(c^*) < 0$, which is equivalent to $LB(c^*) < 0$. Under Rule 2, any core with $w_{\text{curr}} \geq w_{\text{wake}}$ has $LB(c) \geq C_{\text{switch}} > 0 > LB(c^*)$, so $c^* \neq c$. Under Rule 3, any core with $R(c) < \delta_{\min}$ has $LB(c) > 0 > LB(c^*)$, so $c^* \neq c$. Under Rule 4, if $LB(c) \geq UB_{\text{best}}$, then candidate c has net cost strictly worse than or equal to an already discovered admissible candidate. Therefore, pruning never discards c^*. □

D. Competitive Ratio Analysis

We now prove that the preemption predicate guarantees bounded performance relative to an offline optimal schedule.

Theorem 2 (Competitive Ratio of OPTIMA Preemption Guard)**.** *Let $\mathcal{S}_{\text{ON}}$ be the schedule generated by OPTIMA under the online Branch-and-Bound preemption predicate:*

$$\begin{aligned}\textit{Preempt } c^* \iff c^* = \arg\min_{c \in \mathcal{P}} LB(c) \\ \textit{and } LB(c^*) < 0\end{aligned} \tag{40}$$

and let $\mathcal{S}_{\text{OPT}}$ be the offline optimal schedule. Then the competitive ratio is bounded by:

$$\frac{\Phi(\mathcal{S}_{\text{ON}})}{\Phi(\mathcal{S}_{\text{OPT}})} \leq 1 + \epsilon \tag{41}$$

where $\epsilon \triangleq \dfrac{C_{\text{switch}}}{\tau_{\min} \cdot w_{\min}} \ll 1$.

Proof. We construct a continuous potential function $\Psi(t)$ to bound the divergence between the online cost $\Phi(\mathcal{S}_{\text{ON}})$ and the offline optimal cost $\Phi(\mathcal{S}_{\text{OPT}})$. Define:

$$\Psi(t) \triangleq \sum_{J_j \in \mathcal{Q}_{\text{ON}}(t)} w_j \cdot \Big(t - r_j\Big) + \sum_{c \in \mathcal{P}} \mathbf{1}_{\{c \text{ is preempted}\}} \cdot C_{\text{switch}} \tag{42}$$

where $\mathcal{Q}_{\text{ON}}(t)$ is the active queue backlog under $\mathcal{S}_{\text{ON}}$.

Consider any preemption event occurring at time t on core c^*:

1) By the preemption predicate (40), preemption occurs if and only if $LB(c^*) < 0$, which implies:
$$(w_{\text{wake}} - w_{\text{curr}}(c^*))R(c^*) > C_{\text{switch}} \tag{43}$$
2) The immediate reduction in total weighted delay achieved by executing J_{wake} rather than waiting is:
$$\Delta\Phi_{\text{gain}} = w_{\text{wake}}R(c^*) \tag{44}$$
3) The delay penalty imposed on the displaced task J_{curr} is $w_{\text{curr}}R(c^*)$, and the switching overhead is C_{switch}. The net online cost change satisfies:
$$\begin{aligned}\Delta\Phi_{\text{ON}} &= w_{\text{curr}}R(c^*) + C_{\text{switch}} - w_{\text{wake}}R(c^*) \\ &= LB(c^*) < 0\end{aligned} \tag{45}$$

Now consider the worst-case adversary: the offline optimal schedule $\mathcal{S}_{\text{OPT}}$ possesses complete future knowledge of all task release times. If $\mathcal{S}_{\text{OPT}}$ does not preempt at time t because a shorter task arrives at $t+\delta$, the maximum extra cost incurred by $\mathcal{S}_{\text{ON}}$ due to preemption is strictly bounded by C_{switch}.

Because preemption is permitted only when $R(c) \geq \delta_{\min} \geq \tau_{\min}$, the maximum number of preemptions that can occur over any interval of length T on any core is bounded by:
$$N_{\text{preempt}}(T) \leq \frac{T}{\tau_{\min}} \tag{46}$$
Thus, thrashing (chattering) is impossible. Integrating over the entire execution horizon $[0, \infty)$:
$$\begin{aligned}\Phi(\mathcal{S}_{\text{ON}}) &\leq \Phi(\mathcal{S}_{\text{OPT}}) + \sum_{k=1}^{N_{\text{preempt}}} C_{\text{switch}} \\ &\leq \Phi(\mathcal{S}_{\text{OPT}}) + \frac{C_{\text{switch}}}{\tau_{\min}} \int_0^\infty \sum_{J_j \in \mathcal{Q}(t)} 1 \, dt \\ &\leq \Phi(\mathcal{S}_{\text{OPT}}) \left(1 + \frac{C_{\text{switch}}}{\tau_{\min} \cdot w_{\min}}\right)\end{aligned} \tag{47}$$
Setting $\epsilon = \frac{C_{\text{switch}}}{\tau_{\min} \cdot w_{\min}}$ completes the proof. □

IV. Virtual Deadline Calculus and Queue Stability via Lyapunov Drift

A. Virtual Deadline Calculus

To maintain Smith's density ordering across continuous scheduling events without requiring global sorting of all system tasks, OPTIMA maintains virtual deadlines in eBPF Dispatch Queues (DSQs).

Definition 4 (Virtual Deadline Update Equation)**.** *When task J_j is enqueued into dispatch queue k at time t, its virtual deadline $v_j(t)$ is computed via:*
$$v_j(t) = \max\left(v_j(t^-),\, t\right) + \tau_j(t) \cdot \frac{100}{\max(w_j, 10)} \tag{48}$$
where $\tau_j(t)$ is the adaptive scheduling quantum computed as:
$$\tau_j(t) = \text{clamp}\left(\frac{Q_{\text{base}}}{1 + q_k(t)},\, \tau_{\min},\, \tau_{\max}\right) \tag{49}$$
Here $q_k(t)$ denotes the current queue depth of target queue k, Q_{base} is the base nominal quantum, and $\tau_{\min}, \tau_{\max}$ are hardware enforcement boundaries.

For interactive tasks ($w_j \geq w_{\text{RT}}$ or $p_j < \tau_{\min}$), $v_j(t) = t$, placing them at the front of the DSQ priority heap. For batch tasks, virtual deadlines advance monotonically in proportion to quantum consumed divided by task weight, exactly preserving Generalized Processor Sharing (GPS) fairness while realizing Smith's WSPT ordering.

B. Queue State Dynamics

We model the system across the four kernel dispatch queues:
$$\begin{aligned}\mathcal{K} &\triangleq \{1, 2, 3, 4\} \\ &\equiv \{\text{DSQ_PERF, DSQ_SHARED_P,} \\ &\qquad \text{DSQ_SHARED_E, DSQ_EFF}\}\end{aligned} \tag{50}$$
Let $\mathbf{Q}(t) = \big(q_1(t), q_2(t), q_3(t), q_4(t)\big)^T \in \mathbb{Z}^4_{\geq 0}$ represent the queue backlog vector at discrete time slot $t \in \mathbb{Z}_{\geq 0}$. The discrete-time queue state evolution equation is:
$$q_k(t+1) = \max\Big(0,\; q_k(t) - s_k(t)\Big) + a_k(t), \quad \forall k \in \mathcal{K} \tag{51}$$
where:

- $a_k(t)$ is the number of task arrivals into queue k during time slot t, with arrival rate $\lambda_k \triangleq \mathbb{E}[a_k(t)]$.
- $s_k(t)$ is the service capacity (number of task dispatches plus work-stealing removals) from queue k during time slot t.

C. Quadratic Lyapunov Function and Drift Analysis

Definition 5 (Quadratic Lyapunov Function)**.** *We define the quadratic Lyapunov function $L(\mathbf{Q}(t))$ as:*
$$L(\mathbf{Q}(t)) \triangleq \frac{1}{2}\sum_{k=1}^{4} q_k(t)^2 = \frac{1}{2}\|\mathbf{Q}(t)\|_2^2 \tag{52}$$

The conditional one-step Lyapunov drift is defined as:
$$\Delta L(\mathbf{Q}(t)) \triangleq \mathbb{E}\Big[L(\mathbf{Q}(t+1)) - L(\mathbf{Q}(t)) \;\Big|\; \mathbf{Q}(t)\Big] \tag{53}$$

Lemma 2 (Lyapunov Drift Upper Bound)**.** *For the queue dynamics in* (51)*, the one-step Lyapunov drift satisfies:*
$$\Delta L(\mathbf{Q}(t)) \leq B - \sum_{k=1}^{4} q_k(t)\, \mathbb{E}\Big[s_k(t) - a_k(t) \;\Big|\; \mathbf{Q}(t)\Big] \tag{54}$$
where B is a finite positive constant defined as:
$$B \triangleq \frac{1}{2}\sum_{k=1}^{4} \mathbb{E}\Big[a_k(t)^2 + s_k(t)^2 \;\Big|\; \mathbf{Q}(t)\Big] < \infty \tag{55}$$

Proof. From (51), for each queue k:
$$q_k(t+1) = \max\Big(0,\; q_k(t) - s_k(t)\Big) + a_k(t) \tag{56}$$

Squaring both sides:

$$q_k(t+1)^2 = \Big(\max\Big(0,\ q_k(t) - s_k(t)\Big) + a_k(t)\Big)^2 = \Big(\max\Big(0,\ q_k(t) - s_k(t)\Big)\Big)^2 + a_k(t)^2 + 2a_k(t)\max\Big(0,\ q_k(t) - s_k(t)\Big) \quad (57)$$

Using the algebraic identity $(\max(0,x))^2 \le x^2$ for all $x \in \mathbb{R}$:

$$\Big(\max\Big(0,\ q_k(t) - s_k(t)\Big)\Big)^2 \le \big(q_k(t) - s_k(t)\big)^2 \quad (58)$$

Furthermore, because $\max(0,x) \le x + \max(0,-x)$ and $a_k(t) \ge 0$:

$$\max\Big(0,\ q_k(t) - s_k(t)\Big) \le q_k(t) \quad (59)$$

Expanding $(q_k(t) - s_k(t))^2$:

$$q_k(t+1)^2 \le q_k(t)^2 - 2q_k(t)s_k(t) + s_k(t)^2 + a_k(t)^2 + 2q_k(t)a_k(t) = q_k(t)^2 + a_k(t)^2 + s_k(t)^2 - 2q_k(t)\big(s_k(t) - a_k(t)\big) \quad (60)$$

Subtracting $q_k(t)^2$ and dividing by 2:

$$\frac{1}{2}\Big(q_k(t+1)^2 - q_k(t)^2\Big) \le \frac{1}{2}\big(a_k(t)^2 + s_k(t)^2\big) - q_k(t)\big(s_k(t) - a_k(t)\big) \quad (61)$$

Summing over all four queues $k \in \{1,2,3,4\}$ and taking conditional expectations given $\mathbf{Q}(t)$:

$$\Delta L(\mathbf{Q}(t)) \le \sum_{k=1}^{4} \frac{1}{2}\mathbb{E}\big[a_k(t)^2 + s_k(t)^2 \mid \mathbf{Q}(t)\big] - \sum_{k=1}^{4} q_k(t)\mathbb{E}\big[s_k(t) - a_k(t) \mid \mathbf{Q}(t)\big] \quad (62)$$

Because arrivals and service capacity per scheduling time slot are physically bounded by the finite core count M and maximum dispatch bandwidth ($a_k(t) \le a_{\max} < \infty$, $s_k(t) \le s_{\max} < \infty$), the constant $B = \frac{1}{2}\sum_{k=1}^{4}\mathbb{E}[a_k(t)^2 + s_k(t)^2 \mid \mathbf{Q}(t)]$ is strictly finite. □

D. Queue Stability and Starvation Freedom

Definition 6 (Admissible Workload Region)**.** *An arrival rate vector $\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \lambda_3, \lambda_4)$ is strictly admissible if there exists $\epsilon_0 > 0$ such that the aggregate workload demand is strictly within the total multiprocessor processing capacity:*

$$\sum_{k=1}^{4} \lambda_k \bar{p}_k \le \Big(M_P c_P + M_E c_E\Big) - \epsilon_0 \quad (63)$$

where $\bar{p}_k$ is the mean nominal execution requirement of tasks arriving in queue k.

Theorem 3 (Strong Queue Stability and Starvation Freedom via Lyapunov Drift)**.** *Under any strictly admissible workload, OPTIMA's adaptive quantum scaling* (49) *and hierarchical work-conserving work-stealing maintain a negative Lyapunov drift:*

$$\Delta L(\mathbf{Q}(t)) \le B - \epsilon \sum_{k=1}^{4} q_k(t) \quad (64)$$

for some strictly positive constant $\epsilon > 0$. Consequently:

1) *The queue backlog process is strongly stable, with bounded average queue depth:*

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \le \frac{B}{\epsilon} < \infty \quad (65)$$

2) *Every task J_j is guaranteed complete starvation freedom, with finite expected waiting time bounded by:*

$$\mathbb{E}[W_j] \le \left(\frac{B}{\epsilon}\right) \cdot \frac{\tau_{\max}}{\min(M_P, M_E)} < \infty \quad (66)$$

Proof. We establish the negative drift condition by examining the service rate $\mathbb{E}[s_k(t) \mid \mathbf{Q}(t)]$. Under OPTIMA's dispatch and work-stealing architecture:

1) **Work-Conservation:** No core $C_k \in \mathcal{M}$ is ever idle while any queue $q_j(t) > 0$. If a core's local and domestic queues are empty, it executes hierarchical cross-domain work-stealing from remote queues. Thus, total system service capacity satisfies $\sum_{k=1}^{4} s_k(t) = \sum_{C_i \in \mathcal{M}} c_i/\bar{p}$.
2) **Adaptive Quantum Scaling:** As queue depth $q_k(t)$ increases, $\tau(q_k) = \max\left(\frac{Q_{\text{base}}}{1+q_k(t)}, \tau_{\min}\right)$ dynamically compresses the time quantum by up to $\frac{Q_{\text{base}}}{\tau_{\min}} = 6\times$. Because each slice expiration triggers a new dispatch, the effective task retirement rate $s_k(t)$ scales inversely with slice duration, accelerating throughput during congestion.

Because the workload is strictly admissible, the aggregate service rate exceeds the arrival rate across all active queues:

$$\mathbb{E}\Big[s_k(t) - a_k(t) \ \Big|\ \mathbf{Q}(t)\Big] \ge \epsilon > 0, \quad \forall k \text{ such that } q_k(t) > 0 \quad (67)$$

Substituting this inequality directly into Lemma 2 yields:

$$\Delta L(\mathbf{Q}(t)) \le B - \sum_{k=1}^{4} q_k(t)\cdot\epsilon = B - \epsilon\sum_{k=1}^{4} q_k(t) \quad (68)$$

By the Foster-Lyapunov Stability Criterion (Neveu's Theorem for irreducible Markov chains), the negative drift condition implies that the queue state process $\mathbf{Q}(t)$ is positive recurrent. Summing (64) over time slots $t = 0, 1, \ldots, T-1$:

$$\mathbb{E}[L(\mathbf{Q}(T))] - \mathbb{E}[L(\mathbf{Q}(0))] \le TB - \epsilon\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \quad (69)$$

Since $L(\mathbf{Q}(T)) \ge 0$:

$$\frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \le \frac{B}{\epsilon} + \frac{\mathbb{E}[L(\mathbf{Q}(0))]}{\epsilon T} \quad (70)$$

Taking the limit superior as $T \to \infty$ establishes strong queue stability:

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{4}\mathbb{E}[q_k(t)] \le \frac{B}{\epsilon} < \infty \quad (71)$$

To prove starvation freedom, observe that by Definition 4, any task J_j enqueued at time t has virtual deadline:

$$v_j(t) \leq t + \tau_{\max} \cdot \frac{100}{10} = t + 10\tau_{\max} \tag{72}$$

Since all newly arriving tasks $J_{j'}$ arriving after $t + 10\tau_{\max}$ receive $v_{j'}(t') \geq t' > v_j(t)$, task J_j cannot be superseded by more than a finite number of subsequent arrivals. Because total expected backlog is bounded by B/ϵ, the maximum number of tasks scheduled ahead of J_j is strictly bounded, establishing that $\mathbb{E}[W_j] \leq \left(\frac{B}{\epsilon}\right) \frac{\tau_{\max}}{\min(M_P, M_E)} < \infty$. □

V. Algorithmic Specifications

We formalize OPTIMA's three core scheduling operations implemented in Linux `sched_ext` eBPF: CPU selection, queue insertion, and work-conserving dispatch.

Algorithm 1 OPTIMA-SELECT-CPU (Synchronous IPC Fast-Path & Asymmetric Domain Selection)

Require: Task p, previous CPU $prev_cpu$, wakeup flags $wake_flags$
Ensure: Selected target CPU $target_cpu$

$tctx \leftarrow$ lookup_task_ctx(p)
if p is pinned OR p is kernel thread **then**
    Dispatch directly to $prev_cpu$ via `SCX_DSQ_LOCAL`; **return** $prev_cpu$
end if
$this_cpu \leftarrow$ smp_processor_id(), $idle_smtmask \leftarrow$ scx_bpf_get_idle_smtmask(), $prev_is_perf \leftarrow$ is_perf_core($prev_cpu$)
if $wake_flags$ & SCX_WAKE_SYNC **then**
    if $this_cpu \in p$.cpus_allowed **and** dsq_nr_queued($this_cpu$) $== 0$ **then**
        goto direct_dispatch with $target_cpu \leftarrow this_cpu$
    end if
    $this_sib \leftarrow$ get_smt_sibling($this_cpu$)
    if $this_sib \geq 0$ **and** $this_sib \in p$.cpus_allowed **and** dsq_nr_queued($this_sib$) $== 0$ **then**
        test_and_clear_idle($this_sib$); **goto** direct_dispatch with $target_cpu \leftarrow this_sib$
    end if
    if $prev_cpu \in p$.cpus_allowed **and** test_and_clear_idle($prev_cpu$) **then**
        goto direct_dispatch with $target_cpu \leftarrow prev_cpu$
    end if
end if
if $prev_is_perf$ **and** $prev_cpu \in p$.cpus_allowed **and** $prev_cpu \in idle_smtmask$ **and** test_and_clear_idle($prev_cpu$) **then**
    goto direct_dispatch with $target_cpu \leftarrow prev_cpu$
end if
if $tctx.core_type \neq$ CORE_TYPE_EFF **then**
    $c \leftarrow$ pick_idle_domestic_core(p, is_perf $=$ TRUE, $idle_smtmask$)
    if $c \geq 0$ **then**
        goto direct_dispatch with $target_cpu \leftarrow c$
    end if
    $c \leftarrow$ pick_idle_domestic_cpu(p, is_perf $=$ TRUE)
    if $c \geq 0$ **then**
        goto direct_dispatch with $target_cpu \leftarrow c$
    end if
end if
if $prev_is_perf$ **and** $p \in p$.cpus_allowed **then**
    if dsq_nr_queued(DSQ_SHARED_P) > 0 **or** ($tctx.is_batch \wedge tctx.avg_runtime \geq$ 3ms) **then**
        $e \leftarrow$ pick_idle_domestic_core(p, is_perf $=$ FALSE, $idle_smtmask$)
        if $e \geq 0$ **then**
            goto direct_dispatch with $target_cpu \leftarrow e$
        end if
    end if
    $tctx.dispatch_local \leftarrow$ FALSE; **return** $prev_cpu$
end if
$e \leftarrow$ pick_idle_domestic_core(p, is_perf $=$ FALSE, $idle_smtmask$)
if $e \geq 0$ **then**
    goto direct_dispatch with $target_cpu \leftarrow e$
end if
if $tctx.weight \geq w_{\mathrm{RT}}$ **then**
    $bb_cpu \leftarrow$ BRANCH-AND-BOUND-PREEMPT($p, tctx$)
    if $bb_cpu \geq 0$ **then**
        $tctx.dispatch_local \leftarrow$ TRUE; **return** bb_cpu
    end if
end if
$tctx.dispatch_local \leftarrow$ FALSE; **return** $prev_cpu$
direct_dispatch:
Compute slice $\tau \in [\tau_{\min}, \tau_{\max}]$ based on $tctx.is_batch$ and $tctx.avg_runtime$
scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, τ, 0)
return $target_cpu$

Algorithm 2 OPTIMA-ENQUEUE (Smith's WSPT Virtual Deadline Calculus & Preemption Guard)

Require: Task p, enqueue flags enq_flags
$tctx \leftarrow$ lookup_task_ctx(p)
$task_cpu \leftarrow$ scx_bpf_task_cpu(p)
if p is pinned OR p is kernel thread OR $tctx.dispatch_local$ **then**
  $tctx.dispatch_local \leftarrow$ FALSE
  scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, $\tau_{\min}$, enq_flags); **return**
end if
{Dantzig Knapsack Target DSQ Selection}
if $tctx.weight \geq w_{\mathrm{RT}}$ **or** $tctx.core_type ==$ CORE_TYPE_PERF **then**
  $target_dsq \leftarrow$ DSQ_PERF
else if $tctx.core_type ==$ CORE_TYPE_EFF **then**
  $target_dsq \leftarrow$ DSQ_EFF
else
  $target_dsq \leftarrow$ is_perf_core($task_cpu$) ? DSQ_SHARED_P : DSQ_SHARED_E
end if
$is_rt \leftarrow (tctx.weight \geq w_{\mathrm{RT}} \vee \neg tctx.is_batch \vee tctx.avg_runtime < 1\text{ms})$
$now \leftarrow$ scx_bpf_now()
if is_rt **then**
  $\tau \leftarrow \tau_{\min}, \quad deadline \leftarrow now$
else
  if $tctx.avg_runtime \geq 3$ms **then**
    $\tau \leftarrow \tau_{\max}$
  else
    $q \leftarrow$ scx_bpf_dsq_nr_queued($target_dsq$)
    $\tau \leftarrow \max\left(\frac{Q_{\mathrm{base}}}{1+q},\ \tau_{\min}\right)$
  end if
  $v \leftarrow \mathrm{clamp}(p.\text{scx.dsq_vtime}, now, now + 2\tau)$
  $w \leftarrow \max(tctx.weight, 10)$
  $\Delta v \leftarrow \mathrm{clamp}\left(\frac{\tau \cdot 100}{w},\ \tau_{\min},\ \tau_{\max}\right)$
  $deadline \leftarrow v + \Delta v$
end if
p.scx.dsq_vtime $\leftarrow deadline$
scx_bpf_dsq_insert_vtime($p, target_dsq, \tau, deadline, enq_flags$)
{Branch-and-Bound Preemption Kick}
$idle_cpu \leftarrow$ scx_bpf_pick_idle_cpu(p.cpus_allowed, 0)
if $idle_cpu \geq 0$ **then**
  scx_bpf_kick_cpu($idle_cpu$, SCX_KICK_IDLE); **return**
end if
if is_rt **then**
  $bb_cpu \leftarrow$ BRANCH-AND-BOUND-PREEMPT($p, tctx$)
  if $bb_cpu \geq 0$ **then**
    scx_bpf_kick_cpu(bb_cpu, SCX_KICK_PREEMPT)
  end if
end if

Algorithm 3 OPTIMA-DISPATCH (Asymmetric Hierarchical Work-Stealing)

Require: Executing core cpu, previous task $prev$
$is_perf \leftarrow$ is_perf_core(cpu)
if is_perf **then**
  {Priority Tier 1: Real-time and latency-critical tasks}
  if scx_bpf_dsq_move_to_local(DSQ_PERF, 0) **then**
    return
  end if
  {Priority Tier 2: Domestic P-core shared queue}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_P, 0) **then**
    return
  end if
  {Priority Tier 3: Cross-domain work-stealing from E-domain}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_E, 0) **then**
    return
  end if
  {Priority Tier 4: Throughput steal from batch queue}
  if scx_bpf_dsq_move_to_local(DSQ_EFF, 0) **then**
    return
  end if
else
  {Priority Tier 1: Domestic E-core shared queue}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_E, 0) **then**
    return
  end if
  {Priority Tier 2: Throughput batch queue}
  if scx_bpf_dsq_move_to_local(DSQ_EFF, 0) **then**
    return
  end if
  {Priority Tier 3: Cross-domain steal from P-domain}
  if scx_bpf_dsq_move_to_local(DSQ_SHARED_P, 0) **then**
    return
  end if
  {Priority Tier 4: Work-conserving safety steal from RT queue}
  if scx_bpf_dsq_move_to_local(DSQ_PERF, 0) **then**
    return
  end if
end if